EXP NO: 5

CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION WITH PCA

AIM:

To demonstrate the application of Unsupervised Learning models, specifically K-Means clustering for grouping data points and Principal Component Analysis (PCA) for dimensionality reduction and visualization, using a suitable dataset.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

[2]:
```
X, y = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=0.60,
    random_state=42
)

df_kmeans = pd.DataFrame(
    X,
    columns=['Feature_1', 'Feature_2']
)

print("Original K-Means Dataset Head:")
print(df_kmeans.head())
```

```
Original K-Means Dataset Head:
   Feature_1  Feature_2
0  -7.155244  -7.390016
1  -7.395875  -7.110843
2  -2.015671   8.281780
3   4.509270   2.632436
4  -8.102502  -7.484961
```

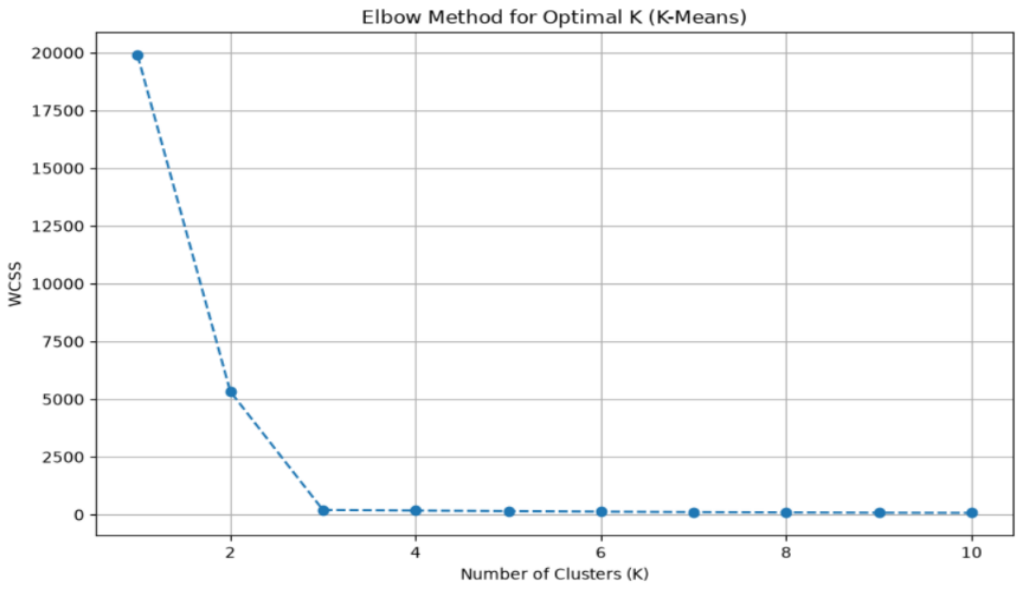
[3]:
```
wcss = []

for i in range(1, 11):
    kmeans = KMeans(
        n_clusters=i,
        init='k-means++',
        max_iter=300,
        n_init=10,
        random_state=42
    )

    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')

plt.title('Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```



[4]:
```python
optimal_k = 3

kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=42
)

clusters = kmeans.fit_predict(X)

df_kmeans['Cluster'] = clusters

print("Dataset with Cluster Labels:")
print(df_kmeans.head())
```

```
Dataset with Cluster Labels:
   Feature_1  Feature_2  Cluster
0  -7.155244  -7.390016        1
1  -7.395875  -7.110843        1
2  -2.015671   8.281780        0
3   4.509270   2.632436        2
4  -8.102502  -7.484961        1
```

[5]:
```python
plt.figure(figsize=(10, 8))

sns.scatterplot(
    x='Feature_1',
    y='Feature_2',
    hue='Cluster',
    data=df_kmeans,
    palette='viridis',
    s=100,
    alpha=0.8
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X',
    label='Centroids'
)

plt.title(f'K-Means Clustering with K={optimal_k}')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```

[6]:
```
silhouette_avg = silhouette_score(X, clusters)

print(
    f"Silhouette Score for K-Means "
    f"(K={optimal_k}): {silhouette_avg:.3f}"
)
```

```
Silhouette Score for K-Means (K=3): 0.908
```

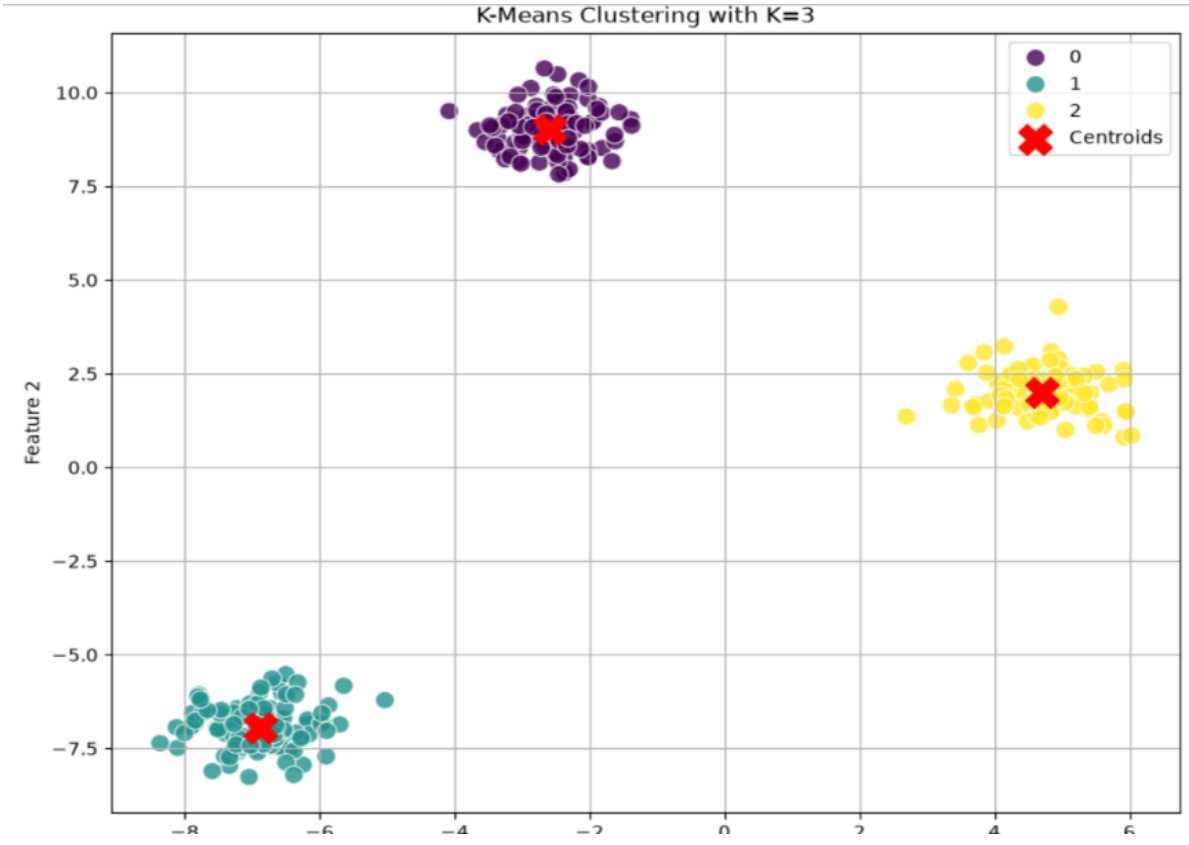
[7]:
```
X_pca, y_pca = make_blobs(
    n_samples=500,
    n_features=4,
    centers=4,
    cluster_std=1.0,
    random_state=25
)

df_pca_original = pd.DataFrame(
    X_pca,
    columns=[f'Feature_{i+1}' for i in range(X_pca.shape[1])]
)

df_pca_original['True_Cluster'] = y_pca

print("Original PCA Dataset Head:")
print(df_pca_original.head())

print("\nOriginal PCA Dataset Shape:")
print(df_pca_original.shape)
```



```
Original PCA Dataset Head:
   Feature_1  Feature_2  Feature_3  Feature_4  True_Cluster
0  -0.638667   1.110057  -6.400722  -0.204990             3
1  -2.951556  -7.657445   3.844794   0.903589             1
2  -0.253177   2.125103  -7.869801   0.559678             3
3  -2.151209   3.401400  -5.734930   0.965230             3
4  -2.347519  -7.230467   3.478891  -0.443440             1

Original PCA Dataset Shape:
(500, 5)
```

[8]:
```
scaler = StandardScaler()

X_pca_scaled = scaler.fit_transform(X_pca)

print("Dataset standardized successfully.")
print("Standardized Dataset Shape:", X_pca_scaled.shape)
```

```
Dataset standardized successfully.
Standardized Dataset Shape: (500, 4)
```

[9]:
```
pca = PCA(n_components=2)

principal_components = pca.fit_transform(X_pca_scaled)

df_principal_components = pd.DataFrame(
    principal_components,
    columns=['Principal_Component_1', 'Principal_Component_2']
)

df_principal_components['True_Cluster'] = y_pca

explained_variance = pca.explained_variance_ratio_

print("Principal Components Head:")
```

```
print("Principal Components Head:")
print(df_principal_components.head())

print("\nExplained Variance Ratio:", explained_variance)

print(
    f"Total Explained Variance by 2 PCs: "
    f"{explained_variance.sum():.3f}"
)
```

```
Principal Components Head:
   Principal_Component_1  Principal_Component_2  True_Cluster
0               0.455305               1.623917             3
1              -2.705622               0.375012             1
2               0.810234               1.966926             3
3               0.427139               2.149626             3
4              -2.407508               0.099250             1

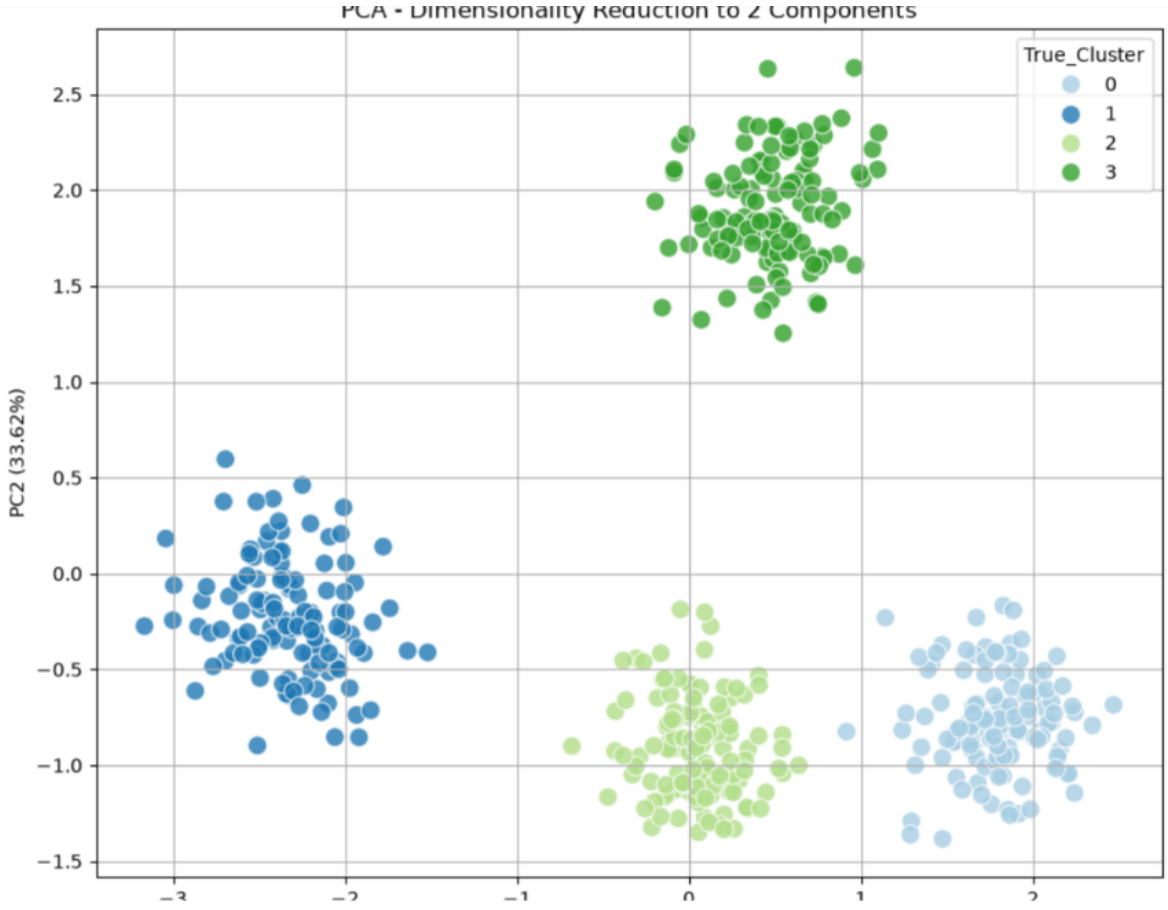
Explained Variance Ratio: [0.57208431 0.33622342]
Total Explained Variance by 2 PCs: 0.908
```

[10]:
```
plt.figure(figsize=(10, 8))

sns.scatterplot(
    x='Principal_Component_1',
    y='Principal_Component_2',
    hue='True_Cluster',
    data=df_principal_components,
    palette='Paired',
    s=100,
    alpha=0.8
)

plt.title('PCA - Dimensionality Reduction to 2 Components')

plt.xlabel(
```



11]:

```
kmeans_pca = KMeans(
    n_clusters=4,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=42
)

clusters_pca = kmeans_pca.fit_predict(principal_components)

df_principal_components['KMeans_Cluster_on_PCA'] = clusters_pca

print("K-Means Clusters on PCA-Reduced Data:")
print(df_principal_components.head())
```

```
K-Means Clusters on PCA-Reduced Data:
   Principal_Component_1  Principal_Component_2  True_Cluster  \
0               0.455305               1.623917             3
1              -2.705622               0.375012             1
2               0.810234               1.966926             3
3               0.427139               2.149626             3
4              -2.407508               0.099250             1

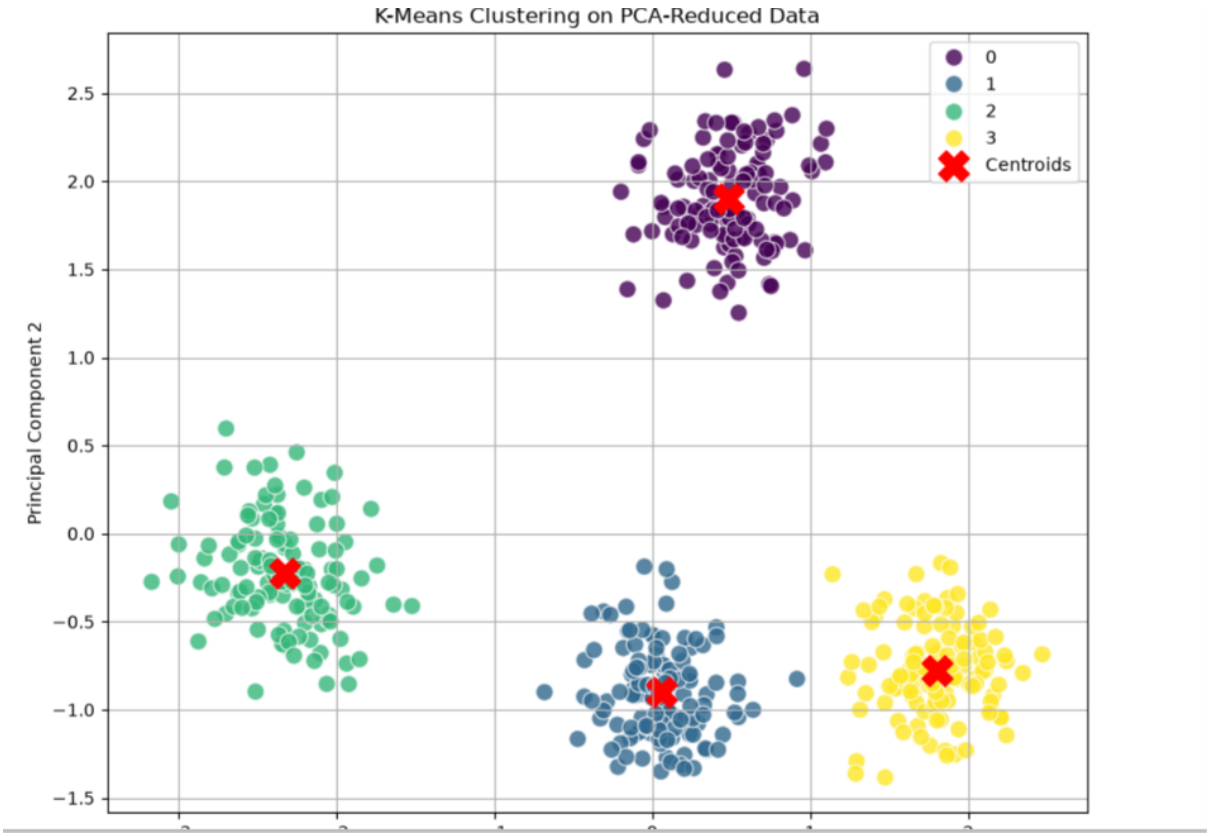
   KMeans_Cluster_on_PCA
0                      0
1                      2
2                      0
3                      0
4                      2
```

[12]:
```
plt.figure(figsize=(10, 8))

sns.scatterplot(
    x='Principal_Component_1',
    y='Principal_Component_2',
    hue='KMeans_Cluster_on_PCA',
    data=df_principal_components,
    palette='viridis',
    s=100,
    alpha=0.8
)

plt.scatter(
    kmeans_pca.cluster_centers_[:, 0],
    kmeans_pca.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X',
    label='Centroids'
)

plt.title('K-Means Clustering on PCA-Reduced Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid(True)
plt.show()
```



[13]:
```
silhouette_avg_pca = silhouette_score(
    principal_components,
    clusters_pca
)

print(
    f"Silhouette Score for K-Means on PCA-Reduced Data "
    f"(K=4): {silhouette_avg_pca:.3f}"
)
```

```
Silhouette Score for K-Means on PCA-Reduced Data (K=4): 0.776
```

[14]:
```
print("EXPERIMENT 5 - K-MEANS AND PCA")
print("--------------------------------")

print("Original K-Means Dataset Shape:", X.shape)
print("Original PCA Dataset Shape:", X_pca.shape)
print("PCA-Reduced Dataset Shape:", principal_components.shape)

print(f"\nK-Means Silhouette Score: {silhouette_avg:.3f}")
print(f"PCA K-Means Silhouette Score: {silhouette_avg_pca:.3f}")

print(
    f"\nTotal Explained Variance by PCA: "
    f"{explained_variance.sum():.3f}"
)

print("\nResult:")
print("K-Means clustering and PCA dimensionality reduction")
print("were implemented and visualized successfully.")
```

```
EXPERIMENT 5 - K-MEANS AND PCA
--------------------------------
Original K-Means Dataset Shape: (300, 2)
```

```
EXPERIMENT 5 - K-MEANS AND PCA
--------------------------------
Original K-Means Dataset Shape: (300, 2)
Original PCA Dataset Shape: (500, 4)
PCA-Reduced Dataset Shape: (500, 2)

K-Means Silhouette Score: 0.908
PCA K-Means Silhouette Score: 0.776

Total Explained Variance by PCA: 0.908

Result:
K-Means clustering and PCA dimensionality reduction
were implemented and visualized successfully.
```

[ ]:

RESULT:

The K-Means clustering and Principal Component Analysis (PCA) techniques were successfully implemented on the given dataset.

• K-Means Clustering effectively grouped the data into distinct clusters based on feature similarity, minimizing intra-cluster distance and maximizing inter-cluster

separation.

• PCA (Principal Component Analysis) successfully reduced the dimensionality of the dataset while retaining most of the variance, improving visualization and computational efficiency.

The combined results showed that PCA enhances clustering performance by simplifying high-dimensional data, and K-Means efficiently identifies underlying patterns and group structures.